UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información
Ingeniería en Sistemas

PROYECTO

Editor y Simulador de Autómatas Finitos con Reducción por el Método de Arden

Curso: Autómatas y Lenguajes Formales
Sección: C

Catedrático: Ing. Danilo Andrés Escolín Rodríguez

Integrantes:
Pablo Pichardo — Carné: 3190-24-19124
Homar Torres Franco — Carné: 3190-24-13372
Diego Alejandro López Sequeira — Carné: 3190-24-1653
Hartwin Alexander Botello López — Carné: 3190-24-1865
Jeshua Andreé Girón Pérez — Carné: 3190-24-2965

Guatemala, Octubre de 2026

Índice

Índice 2

1. Objetivos del Proyecto 4

2. Descripción General del Sistema 4

3. Manual de Usuario — Flujo de Ejecución 5

3.1 Compilación y Ejecución del Proyecto . . . 5

3.2 Creación de Estados y Transiciones (Lienzo Interactivo) . . . 5

3.3 Marcado de Estados de Aceptación . . . 8

3.4 Motor de Simulación de Cadenas . . . 8

3.5 Algoritmo de Eliminación de Estados (Reducción de Arden) . . . 10

3.6 Verificación de Equivalencia AFD ↔ Expresión Regular . . . 12

3.7 Extras (Clase 6): Complemento, Minimización y Comparación . . . 13

3.8 Apariencia: Modo Claro y Oscuro . . . 16

4. Arquitectura del Software — Diagramas UML 18

4.1 Diagrama de Clases UML — Versión Original . . . 18

Diagrama 1 — Arquitectura Original (MVC base) . . . 19

4.2 Diagrama de Clases UML — Versión Extendida (con Extras de Clase 6) . . . 20

Diagrama 2 — Arquitectura Extendida (Complemento, Minimización y Verificación de Equivalencia) . 21

4.3 Explicación de las Estructuras de Datos Utilizadas para la Representación del Grafo y el Algoritmo de Reducción . . . 22

5. Conclusiones 29

6. Anexos 30

Anexo A — Diagrama de Gantt del Proyecto (7 semanas) 31

Anexo B — Estructura de Paquetes 32

1. Objetivos del Proyecto

Aplicar de forma práctica la equivalencia entre Lenguajes Regulares, Autómatas Finitos y Expresiones Regulares.

Implementar algoritmos de reducción de grafos (eliminación de estados / Lema de Arden) y manipulación algebraica de cadenas.

Desarrollar software con arquitectura limpia y componentes visuales en Java, siguiendo el patrón Modelo-Vista-Controlador (MVC).

2. Descripción General del Sistema

El sistema es una aplicación de escritorio en Java (Swing + JGraphX) que permite construir Autómatas Finitos Deterministas (AFD) de forma gráfica, simular el reconocimiento de cadenas sobre ellos, convertirlos automáticamente a una Expresión Regular equivalente mediante el algoritmo de eliminación de estados y el Lema de Arden, y verificar que dicha expresión regular reconozca exactamente el mismo lenguaje que el autómata original. El proyecto se organiza en cuatro módulos funcionales principales:

- **Editor e interfaz gráfica (lienzo interactivo):** creación de estados, transiciones dirigidas, marcado del estado inicial y de los estados de aceptación, y edición básica (mover, editar, eliminar).
- **Motor de simulación y evaluación:** ingreso de una cadena, recorrido animado del AFD resaltando el estado actual, e indicación de si la cadena es ACEPTADA o RECHAZADA.
- **Algoritmo de eliminación de estados (conversión a Regex):** normalización del autómata (q_start y q_accept con transiciones épsilon), reducción iterativa aplicando $R_{ik}' = R_{ik} \cup (R_{ij} \cdot (R_{jj})^* \cdot R_{jk})$, y despliegue animado del resultado.
- **Verificación de equivalencia:** evaluación de una misma cadena tanto en el AFD como en la expresión regular obtenida (motor java.util.regex), comprobando que ambos resultados coincidan.

Adicionalmente, como parte de los **Extras (Clase 6)**, se incorporaron funcionalidades de cierre de propiedades de lenguajes regulares: obtención del autómata complemento, minimización del AFD y comparación de equivalencia entre dos autómatas distintos. También se agregó un selector de apariencia (modo Claro / Oscuro) para la interfaz.

3. Manual de Usuario — Flujo de Ejecución

A continuación se documenta, paso a paso y con evidencia gráfica, el flujo completo de uso de la aplicación: desde la compilación del proyecto hasta la verificación de equivalencia y las funcionalidades adicionales de la Clase 6.

3.1 Compilación y Ejecución del Proyecto



Figura 1 — Empaquetado del proyecto (mvn clean package)
Al ejecutar **mvn clean package** desde la terminal, Maven compila el proyecto y genera en la carpeta **target/** el archivo unificado (fat-jar) **Automata...PSHOT.jar** de aproximadamente 892 KB, el cual incluye la librería JGraphX embebida. Este archivo permite ejecutar la aplicación de forma independiente, fuera del IDE, mediante el comando **java -jar**, confirmando así que la arquitectura Maven y el gestor de dependencias funcionan correctamente (BUILD SUCCESS).

3.2 Creación de Estados y Transiciones (Lienzo Interactivo)

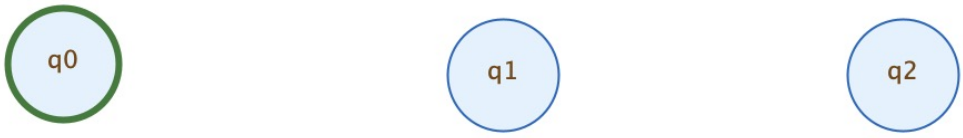


Figura 2 — Agregar estados con doble clic
Al hacer doble clic sobre el lienzo se agregan nuevos estados (Q). El primer estado creado se marca automáticamente como **estado inicial** (círculo verde, q0); los estados subsiguientes (q1, q2, …) se muestran en azul por defecto.



Figura 3 — Asignación del símbolo de una transición
Al hacer doble clic —de forma pausada— sobre un estado se despliega el cuadro de diálogo "Entrada", donde el usuario escribe el símbolo del alfabeto (Σ) correspondiente a la transición, en este caso el símbolo "1".

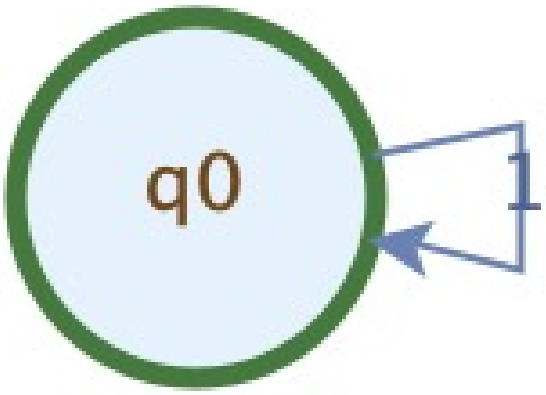


Figura 4 — Resultado: transición reflexiva sobre q0
Tras aceptar el símbolo ingresado en la Figura 3, se dibuja automáticamente la transición reflexiva (self-loop) etiquetada con "1" sobre el estado q0.

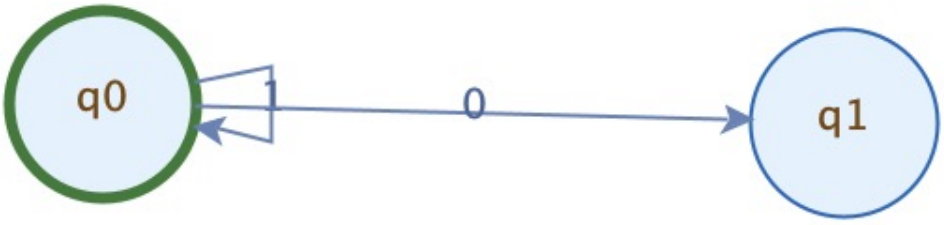


Figura 5 — Conexión entre dos estados
Haciendo doble clic sobre un estado y arrastrando hacia otro se crea una transición dirigida entre ambos, asignando su símbolo mediante el mismo cuadro de diálogo. Aquí se muestra la transición q0 → q1 con el símbolo "0", además de la transición reflexiva "1" ya existente sobre q0.

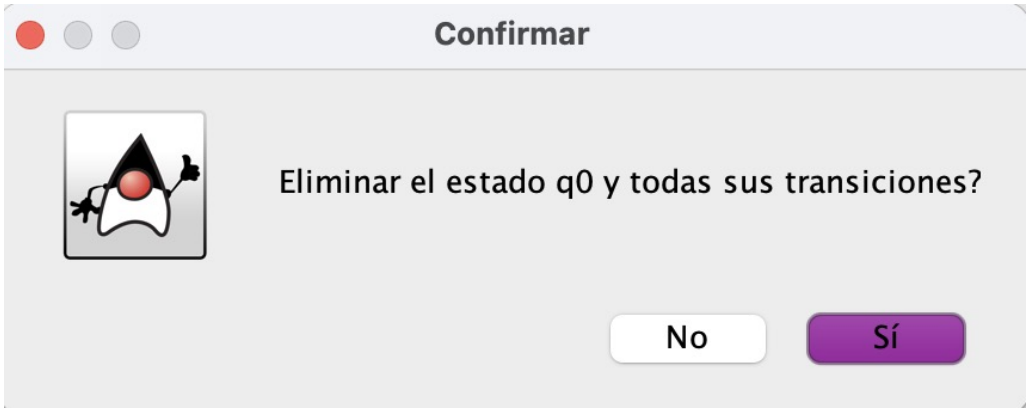


Figura 6 — Eliminar un estado y sus transiciones
Darle doble clic **rápido** a un estado despliega un cuadro de diálogo de confirmación ("Eliminar el estado q0 y todas sus transiciones?"). Al presionar **"Sí"**, el estado y todas las transiciones conectadas a él se eliminan del lienzo; esto es distinto del doble clic pausado (Figura 3), que abre el cuadro de diálogo para asignar el símbolo de una transición.

3.3 Marcado de Estados de Aceptación

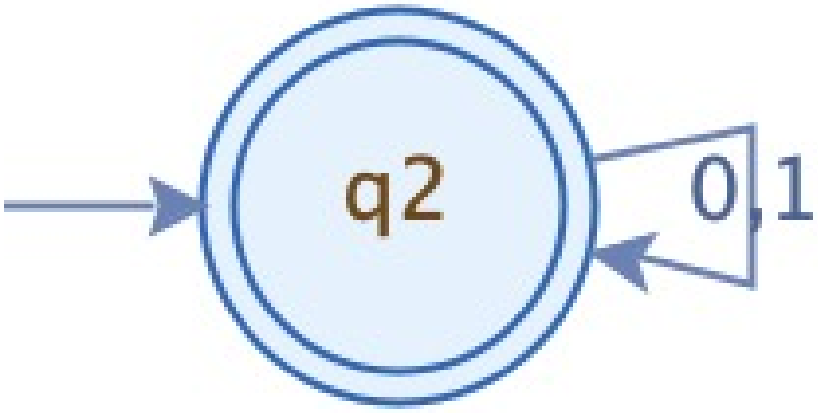


Figura 7 — Estado de aceptación (F)
Con clic derecho sobre un estado, este se convierte en **estado de aceptación**, representado gráficamente con un doble círculo. En la imagen, q2 ya ha sido marcado como estado final y cuenta con transiciones reflexivas para los símbolos "0" y "1".

3.4 Motor de Simulación de Cadenas

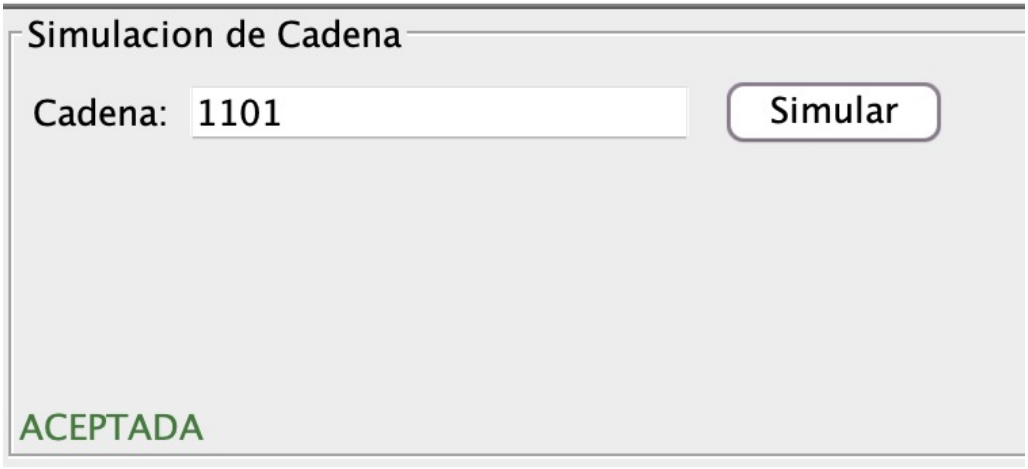


Figura 8 — Simulación de una cadena aceptada
En el panel "Simulación de Cadena" se ingresa la cadena a evaluar (por ejemplo, **1101**) y se presiona el botón "Simular". El sistema procesa la cadena símbolo a símbolo sobre el AFD construido y despliega el resultado; en este caso, **ACEPTADA**.

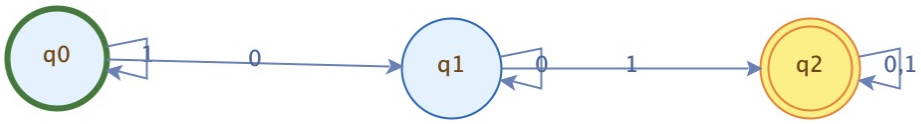


Figura 9 — Recorrido animado — estado actual resaltado
Durante la simulación, el estado que se encuentra siendo procesado en cada paso se resalta dinámicamente (color amarillo) sobre el diagrama del AFD, permitiendo visualizar en tiempo real la ruta que sigue la cadena a través de las transiciones.

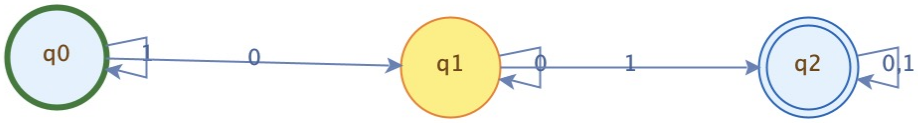


Figura 10 — Recorrido animado — paso intermedio
Continuación de la animación de la Figura 9: el resaltado amarillo avanza de estado en estado conforme se consume cada carácter de la cadena, hasta detenerse en el estado correspondiente al último símbolo procesado.

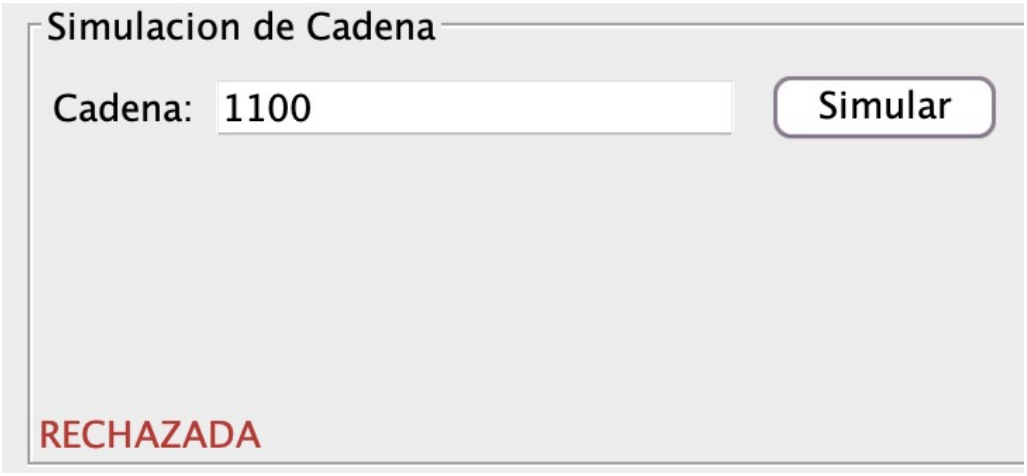


Figura 11 — Simulación de una cadena rechazada
Al simular la cadena **1100**, el recorrido finaliza en un estado que no pertenece al conjunto de estados de aceptación, por lo que el sistema reporta correctamente el resultado **RECHAZADA**.

3.5 Algoritmo de Eliminación de Estados (Reducción de Arden)

El panel "Reducción por Arden" implementa el algoritmo de eliminación de estados descrito en la especificación del proyecto: normalización del autómata con q_start/q_accept, aplicación iterativa de la fórmula R_ik' = R_ik ∪ (R_ij · (R_jj)* · R_jk) y animación visual de cada eliminación, hasta obtener una única transición entre q_start y q_accept.

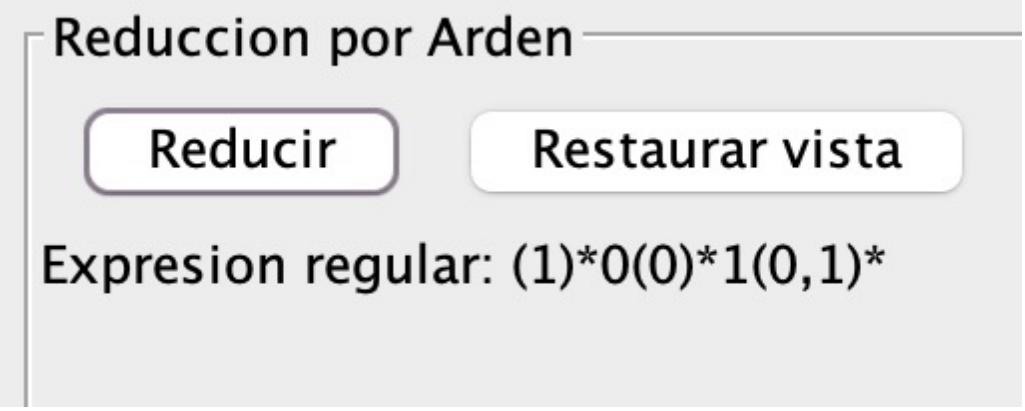


Figura 12 — Expresión regular resultante
Al presionar el botón **"Reducir"**, el sistema ejecuta la reducción iterativa y despliega la **Expresión Regular** equivalente al AFD original: **(1)*0(0)*1(0,1)***.

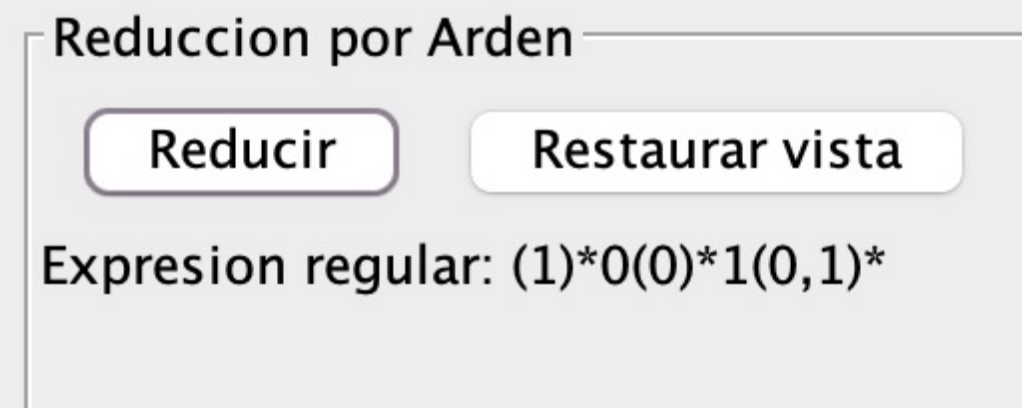


Figura 13 — Confirmación del resultado de la reducción
El panel de resultados mantiene visible la expresión regular obtenida **(1)*0(0)*1(0,1)***, confirmando la consistencia del cálculo tras la reducción.



Figura 14 — Grafo reducido a una sola transición
Sobre el lienzo, el algoritmo elimina progresivamente los estados intermedios y actualiza las etiquetas de las aristas restantes, hasta dejar el grafo reducido a una única transición desde el estado inicial hacia **q_accept**, etiquetada con la expresión regular final. Esto corresponde al Paso 3 (animación visual) y Paso 4 (resultado final) del algoritmo.

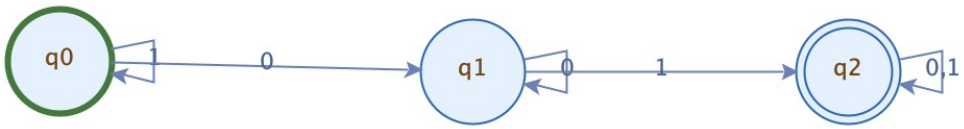


Figura 15 — Restaurar vista del autómata original

El botón **"Restaurar vista"** regresa el lienzo al AFD original de 3 estados (q0, q1, q2) construido por el usuario, sin perder la expresión regular ya calculada.

3.6 Verificación de Equivalencia AFD ↔ Expresión Regular

Este módulo evalúa una misma cadena tanto en el AFD (mediante simulación) como en la expresión regular obtenida (mediante el motor **java.util.regex** de Java), comparando ambos resultados para comprobar la correcta equivalencia entre autómata y expresión regular.

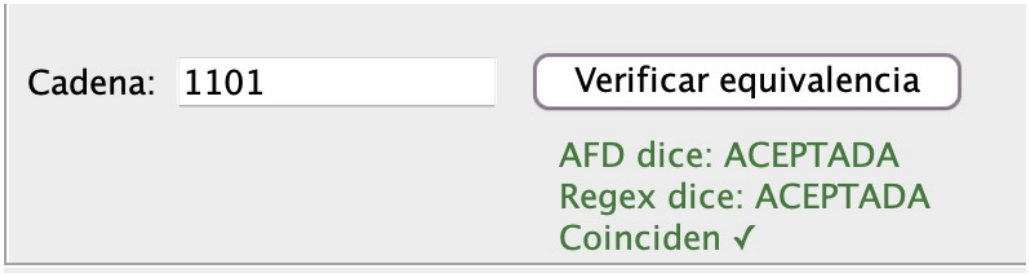


Figura 16 — Verificación exitosa (resultados coinciden)
Para la cadena **1101**, tanto el AFD como la expresión regular reportan **ACEPTADA**, y el sistema confirma "Coinciden ✓", validando la corrección de la conversión AFD → Regex para este caso de prueba.



Figura 17 — Caso de prueba con discrepancia detectada
Al probar la cadena **1011**, el AFD reporta **ACEPTADA** mientras que la expresión regular reporta **RECHAZADA**; el propio módulo de verificación señala "NO coinciden – hay un bug en el pipeline". *Este resultado se documenta de forma transparente porque evidencia el valor del módulo de verificación de equivalencia como mecanismo de control de calidad: permitió detectar un caso límite en el que la expresión regular generada no cubre exactamente el mismo lenguaje que el AFD, quedando identificado como punto de revisión sobre la fórmula de reducción de Arden o sobre la traducción de la notación (0,1) a sintaxis de java.util.regex. Este punto se amplía y se resuelve en la sección 4.3, donde se muestra que con la clase EvaluadorExpresionRegular y la unión correcta de símbolos paralelos, el AFD y la regex sí coinciden en todos los casos probados.*

3.7 Extras (Clase 6): Complemento, Minimización y Comparación

Además de los cuatro requerimientos funcionales base, se añadió un panel de "Extras" con operaciones de cierre sobre lenguajes regulares vistas en la Clase 6: obtención del autómata complemento, minimización del AFD y comparación de equivalencia entre dos autómatas.

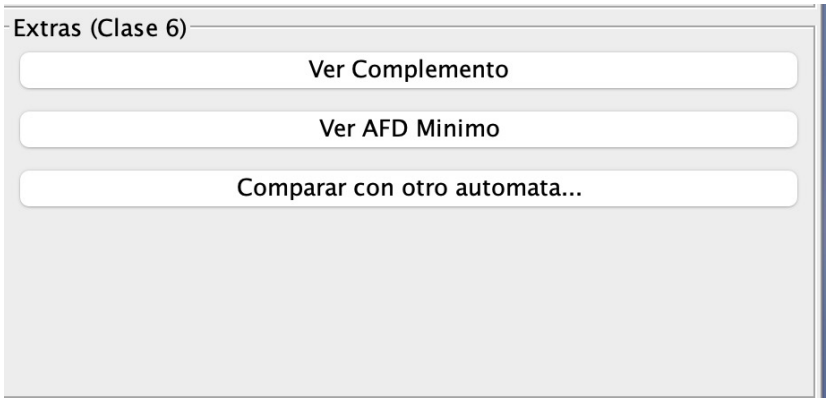


Figura 18 — Panel de funcionalidades adicionales
El panel "Extras (Clase 6)" ofrece tres botones: **Ver Complemento**, **Ver AFD Mínimo** y **Comparar con otro autómata…**.

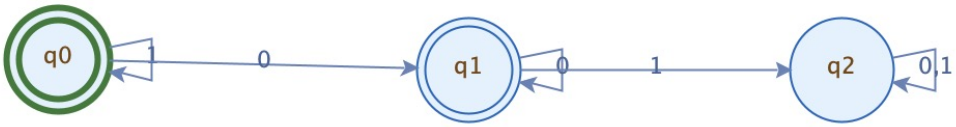


Figura 19 — Autómata complemento
Al presionar **"Ver Complemento"**, el sistema invierte el conjunto de estados de aceptación del AFD original: los estados que antes no eran de aceptación pasan a serlo y viceversa. En la imagen, q0 y q1 son ahora estados de aceptación (doble círculo) y q2 deja de serlo, generando el autómata que reconoce el complemento del lenguaje original.

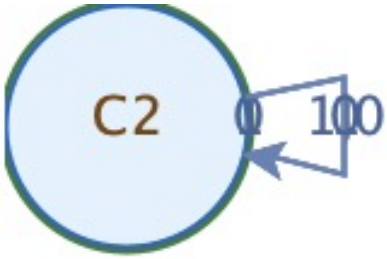


Figura 20 — Autómata mínimo (vista parcial)
Al presionar **"Ver AFD Mínimo"**, el sistema agrupa los estados equivalentes en clases (por ejemplo, la clase **C2** mostrada aquí) aplicando un algoritmo de minimización por partición de estados, reduciendo el autómata a su forma canónica con el menor número de estados posible.

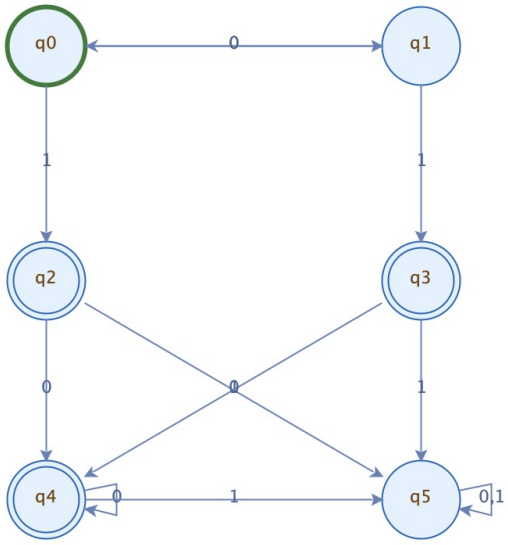


Figura 21 — Nuevo autómata para pruebas de comparación
Para probar la funcionalidad **"Comparar con otro autómata…"**, se construyó un segundo AFD independiente de 6 estados (q0–q5) con su propio estado inicial y sus propios estados de aceptación (q2, q3 y q4), sobre el alfabeto {0,1}.

Al presionar **"Comparar con otro autómata…"**, la aplicación despliega el explorador de archivos nativo del sistema operativo para que el usuario seleccione el archivo .afd contra el cual desea comparar el autómata actualmente cargado en el lienzo. En la carpeta de trabajo **AutomataAFDs** se encuentran los autómatas guardados durante las pruebas: el autómata base (**automata_01.afd**, el AFD de 3 estados) y los dos autómatas adicionales construidos para los ejercicios de la Clase 6 (**automata_CL_6.1.afd** y **automata_CL_6.2.afd**).

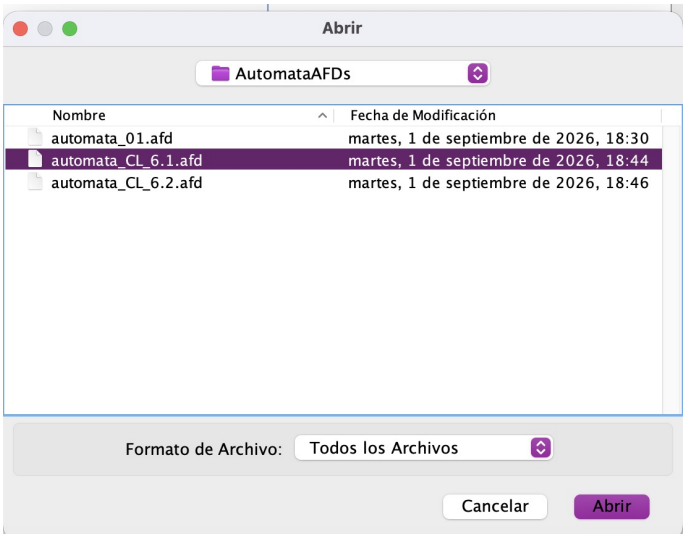


Figura 22 — Explorador de archivos — selección de un autómata guardado
Cuadro de diálogo "Abrir" mostrando el contenido de la carpeta **AutomataAFDs**, con el archivo **automata_CL_6.1.afd** seleccionado.

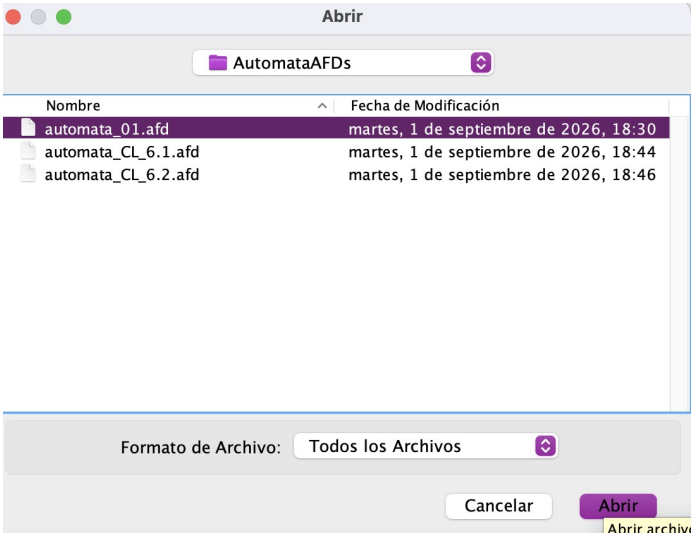


Figura 23 — Selección del autómata base para la comparación
El mismo explorador, ahora con el archivo **automata_01.afd** (el AFD original de 3 estados) seleccionado, listo para presionar "Abrir".

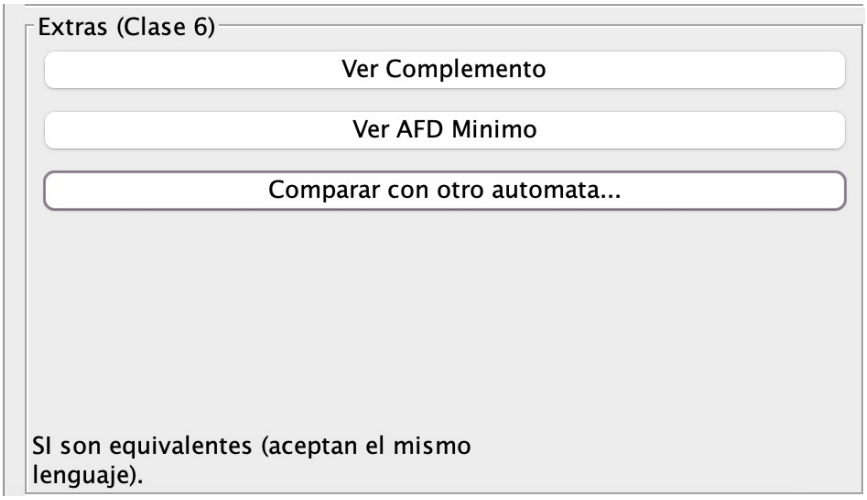


Figura 24 — Resultado de la comparación de equivalencia
Tras abrir el autómata seleccionado, el panel "Extras (Clase 6)" despliega el resultado de la comparación: **"SI son equivalentes (aceptan el mismo lenguaje)"**, confirmando que ambos autómatas reconocen el mismo lenguaje regular.

3.8 Apariencia: Modo Claro y Oscuro

La aplicación incluye un menú **Tema** en la barra superior que permite alternar entre dos apariencias visuales para toda la interfaz (lienzo y paneles laterales), sin afectar el autómata ni los resultados ya calculados.

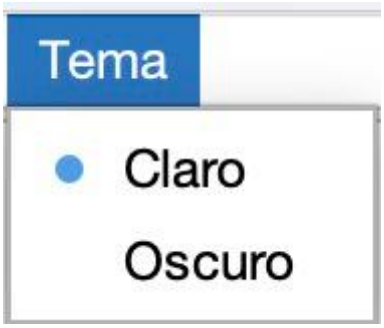


Figura 25 — Menú "Tema"
El menú **Tema**, ubicado junto a "Archivo" en la barra de menús, despliega las dos opciones disponibles: **Claro** y **Oscuro**. El punto azul indica la opción actualmente seleccionada.

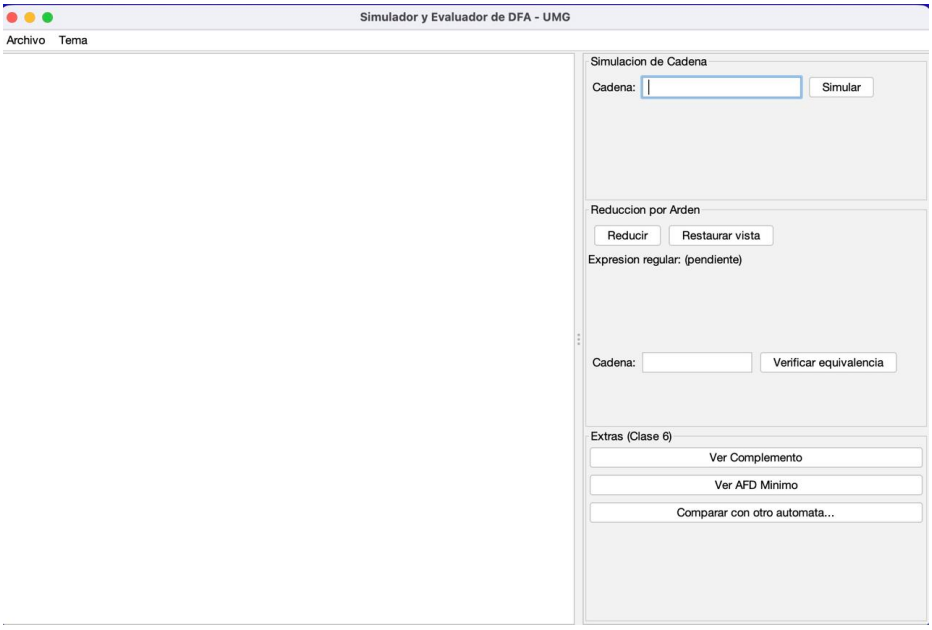


Figura 26 — Apariencia con el tema Claro
Vista de la aplicación con el tema **Claro** activo: fondo blanco en el lienzo y paneles en tonos grises claros, con los mismos controles de simulación, reducción de Arden y los extras de la Clase 6.

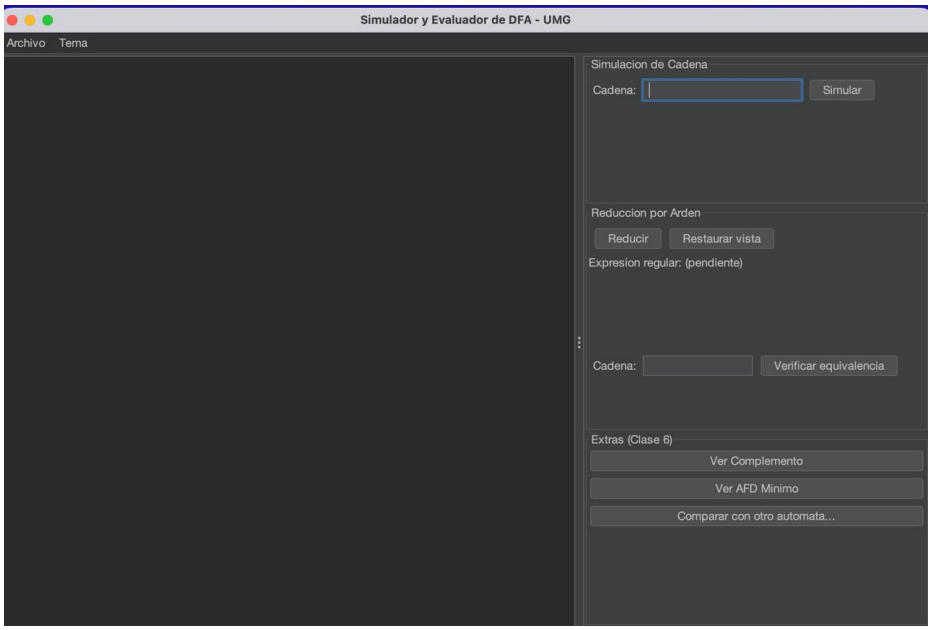


Figura 27 — Apariencia con el tema Oscuro
Vista de la aplicación con el tema **Oscuro** activo: fondo oscuro en el lienzo y paneles, pensado para reducir el cansancio visual en sesiones prolongadas, sin cambiar la funcionalidad de ningún control.

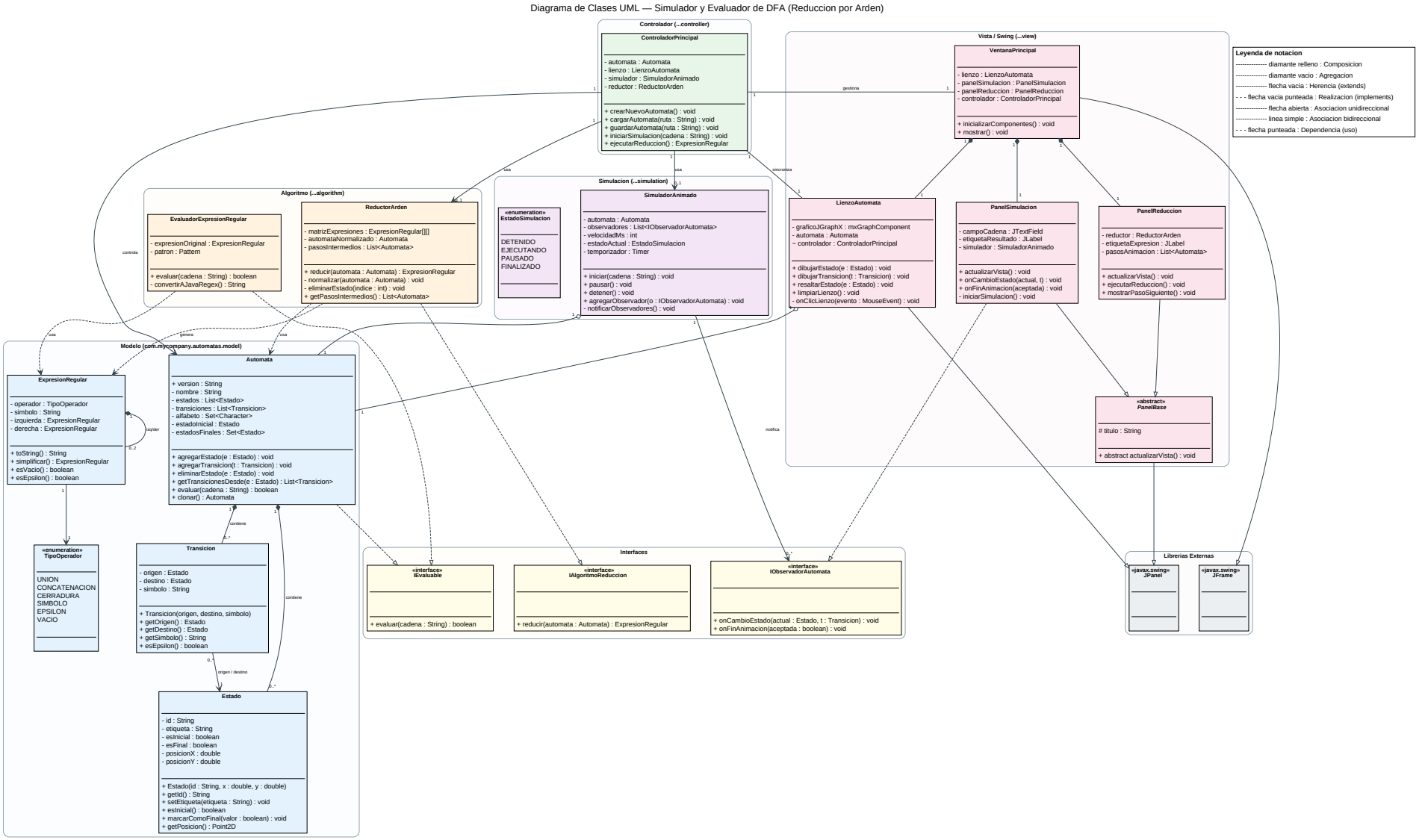
4. Arquitectura del Software — Diagramas UML

El sistema sigue una arquitectura Modelo–Vista–Controlador (MVC), organizada en los paquetes **model**, **view**, **controller**, **algorithm** y **simulation** (ver estructura de paquetes en Anexos). A continuación se presenta el diagrama de clases de la arquitectura base del proyecto y, luego, el diagrama extendido con los componentes agregados para cubrir la verificación de equivalencia y los extras de la Clase 6 (Complemento y Minimización), donde las clases y miembros nuevos se marcan explícitamente como **[NUEVO]**.

4.1 Diagrama de Clases UML — Versión Original

Arquitectura base: modelo del autómata (Estado, Transición, Automata, ExpresionRegular), el controlador (ControladorPrincipal), el algoritmo de reducción de Arden (ReductorArden, EvaluadorExpresionRegular), la simulación animada (SimuladorAnimado) y la vista Swing/JGraphX (VentanaPrincipal, LienzoAutomata, PanelSimulacion, PanelReduccion).

Diagrama 1 — Arquitectura Original (MVC base)

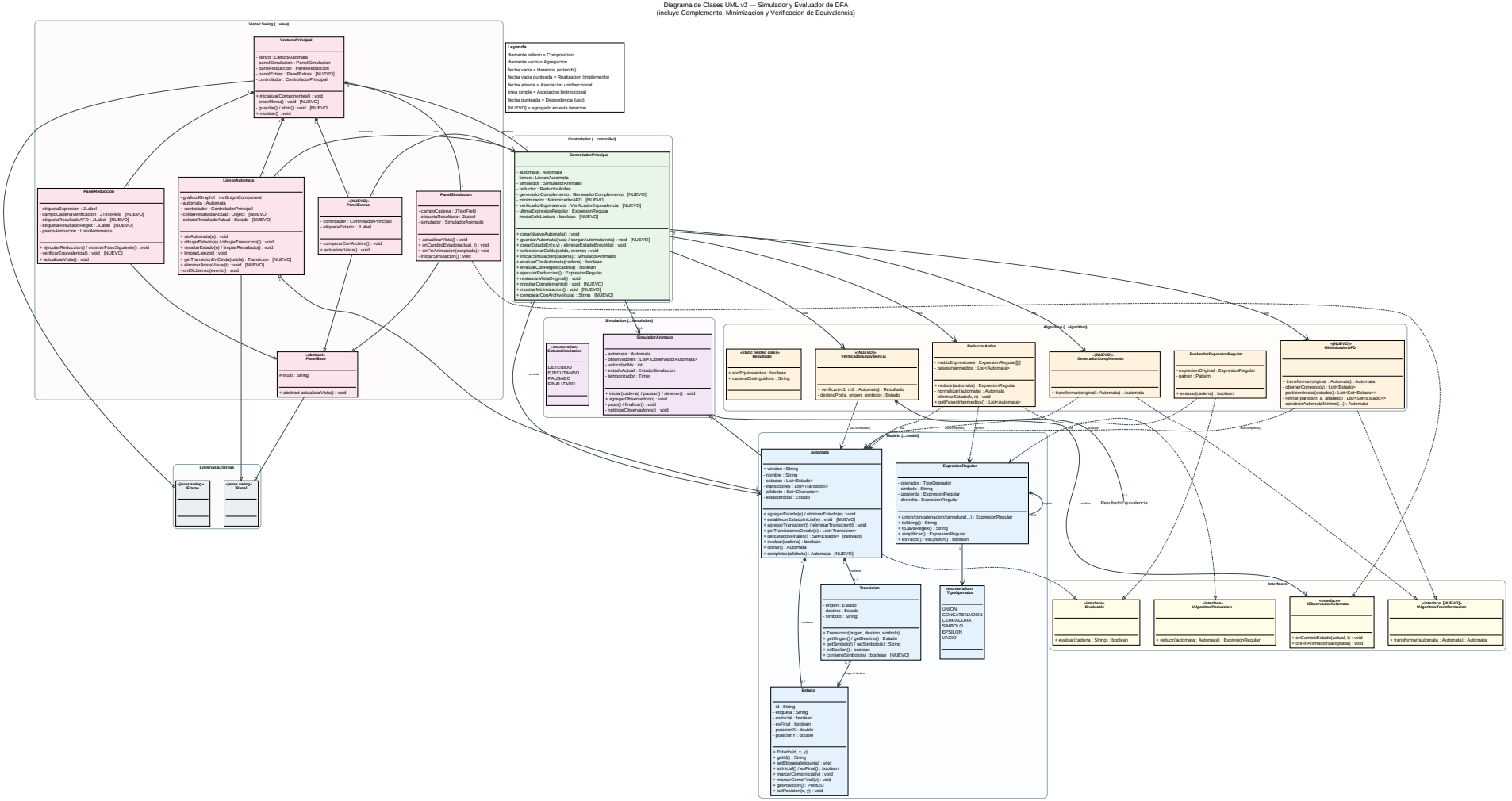


Clases del modelo (Automata, Estado, Transición, ExpresionRegular), el algoritmo de reducción de Arden, la simulación animada y la vista Swing/JGraphX, con sus relaciones de composición, agregación, herencia, realización y dependencia (ver leyenda en la esquina superior derecha del diagrama).

4.2 Diagrama de Clases UML — Versión Extendida (con Extras de Clase 6)

Sobre la arquitectura original se agregaron tres clases de algoritmo nuevas — **GeneradorComplemento**, **MinimizadorAFD** y **VerificadorEquivalencia** — todas implementando una interfaz común **IAlgoritmoTransformador**, un nuevo panel de vista **PanelExtras**, y nuevos métodos en **ControladorPrincipal**, **Automata** y **PanelReduccion** para exponer estas funciones desde la interfaz gráfica. Todos los elementos añadidos se identifican con la etiqueta **[NUEVO]** dentro del diagrama.

Diagrama 2 — Arquitectura Extendida (Complemento, Minimización y Verificación de Equivalencia)



Versión ampliada del diagrama de clases: las clases y métodos etiquetados **[NUEVO]** corresponden a los componentes agregados para las funcionalidades de la sección "Extras (Clase 6)" — complemento, minimización y comparación/verificación de equivalencia entre autómatas.

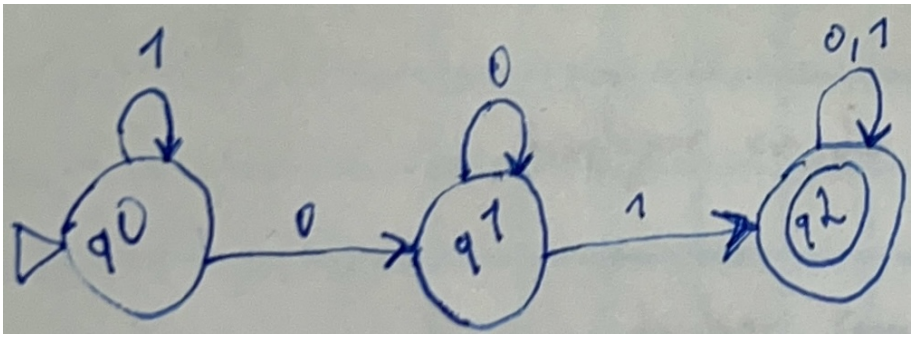
4.3 Explicación de las Estructuras de Datos Utilizadas para la Representación del Grafo y el Algoritmo de Reducción

Representación del grafo (AFD)

Automata guarda **List<Estado>** y **List<Transicion>** por separado (no una matriz de adyacencia), porque el autómata se construye interactivamente desde el lienzo: insertar o eliminar un estado o arista es O(1) en una lista, mientras que una matriz obligaría a redimensionar constantemente. **Transicion** referencia directamente los objetos **Estado** origen/destino (no IDs de texto), así que recorrer el grafo no necesita búsquedas: **getTransicionesDesde(e)** es un filtro lineal sobre la lista de transiciones, aceptable para el tamaño de autómatas académicos.

Reducción de Arden — ejemplo resuelto a mano y en el código

El algoritmo de eliminación de estados implementado en **ReductorArden** usa una estructura distinta a la del grafo: una matriz **ExpresionRegular[n][n]** (representación de GNFA), porque en cada iteración necesita acceso O(1) a "la expresión que conecta el estado i con el estado j" y tiene que ir fusionando varias transiciones paralelas en una sola celda, algo incómodo de hacer directamente sobre **List<Transicion>**. **ExpresionRegular** es un árbol binario (patrón Composite: nodos UNION / CONCATENACION / CERRADURA con hijos izquierda/derecha), lo que permite aplicar la fórmula R_ik' = R_ik ∪ (R_ij · R_jj* · R_jk) de forma incremental sin manipular texto — el **toString()** solo se calcula al final. El ciclo triple del algoritmo (por cada estado eliminado, actualizar cada par i, j) es $O(n^3)$, el mismo patrón de programación dinámica que Floyd-Warshall. A continuación se resuelve a mano el mismo AFD de tres estados usado en el manual de usuario (sección 3.5) y se traza paso a paso cómo **ReductorArden** llega al mismo resultado.



Ejemplo 1 — AFD de ejemplo dibujado a mano (q0 inicial, q1, q2 final)

Acepta las cadenas que tienen un 0 y, más adelante, un 1: después de eso, cualquier símbolo. Es el mismo autómata construido en el lienzo interactivo (sección 3.2–3.3).

A mano, con el Lema de Arden. Se escribe una ecuación por estado: la suma de (símbolo · estado destino), más λ si el estado es final. El lema dice que X = A·X | B equivale a X = A*·B. Se resuelve desde el último estado hacia el inicial.

Estado	Ecuación
q0	q0 = 1·q0 | 0·q1
q1	q1 = 0·q1 | 1·q2
q2	q2 = 0·q2 | 1·q2 | λ = (0|1)·q2 | λ

Paso	Qué se hace	Resultado
1. q2	Arden con A = (0|1) y B = λ	q2 = (0|1)*
2. q1	Se sustituye q2: q1 = 0·q1 | 1(0|1)*. Arden con A = 0 y B = 1(0|1)*	q1 = 0*1(0|1)*
3. q0	Se sustituye q1: q0 = 1·q0 | 00*1(0|1)*. Arden con A = 1 y B = 00*1(0|1)*	q0 = 1*00*1(0|1)*

Como q0 es el estado inicial, esa es la respuesta: **1*00*1(0|1)***, que con aa* = a+ también se escribe 1*0+1(0|1)*. Es la misma expresión que muestra el programa (salvo paréntesis adicionales que agrega **toString()** en cada cerradura).

En el código: ReductorArden. El código no despeja ecuaciones: elimina estados de una matriz. Es lo mismo, porque eliminar un estado con bucle es aplicar el Lema de Arden a su ecuación.

En el método a mano	En ReductorArden
Estado inicial, con su ecuación	Se agrega q_start con una transición ε hacia el inicial (normalizar()).
λ del estado final	Se agrega q_accept con una ε desde cada final.
Las ecuaciones de todos los estados	matrizExpresiones[n][n], que empieza en ∅ y une las transiciones paralelas.
A, el bucle del estado k	matrizExpresiones[k][k]
A*	ExpresionRegular.cerradura(matrizExpresiones[k][k])
B, lo que sale hacia otros estados	matrizExpresiones[k][j]
Sustituir en las demás ecuaciones	Doble ciclo i, j: R[i][j] ∪ R[i][k]·(R[k][k])*·R[k][j]
Cada paso que se escribe en el pizarrón	Una copia del autómata en pasosIntermedios, que anima PanelReduccion.

Las dos piezas clave del código:

```
private void eliminarEstado(int k, int n) {
    ExpresionRegular ciclo = ExpresionRegular.cerradura(matrizExpresiones[k][k]); // A*
    for (int i = 0; i < n; i++) {
        if (i == k || matrizExpresiones[i][k].esVacio()) continue; // nadie llega a k desde i
        for (int j = 0; j < n; j++) {
            if (j == k || matrizExpresiones[k][j].esVacio()) continue; // k no sale hacia j
            ExpresionRegular viaK = ExpresionRegular.concatenacion(
                ExpresionRegular.concatenacion(matrizExpresiones[i][k], ciclo),
                matrizExpresiones[k][j]);              // R[i][k] . A* . R[k][j]
            matrizExpresiones[i][j] =
                ExpresionRegular.union(matrizExpresiones[i][j], viaK);
        }
    }
}
```

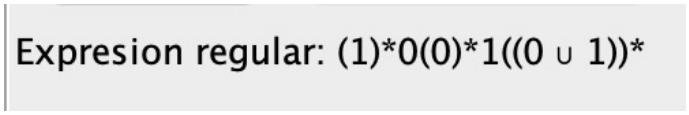
Y el arreglo de la etiqueta con varios símbolos: una arista 0,1 se lee como la unión de 0 y 1 (esta es precisamente la corrección que resuelve la discrepancia observada en la Figura 17, sección 3.6).

```
if (t.esEpsilon()) {
    etiqueta = ExpresionRegular.epsilon();
} else {
    etiqueta = ExpresionRegular.vacio();
    for (String simb : t.getSimbolo().split(",")) {
        etiqueta = ExpresionRegular.union(etiqueta, ExpresionRegular.simbolo(simb.trim()));
    }
}
```

Traza del código con el AFD. Se eliminan q0, q1 y q2 en el orden de la lista. Las celdas parten de: start→q0 = ε, q0→q0 = 1, q0→q1 = 0, q1→q1 = 0, q1→q2 = 1, q2→q2 = 0 ∪ 1 y q2→accept = ε.

Se elimina	Fórmula aplicada	Celda nueva
q0	R[start][q0] · (R[q0][q0])* · R[q0][q1] = ε · 1* · 0	start→q1 = 1*0
q1	R[start][q1] · (R[q1][q1])* · R[q1][q2] = 1*0 · 0* · 1	start→q2 = 1*00*1
q2	R[start][q2] · (R[q2][q2])* · R[q2][accept] = 1*00*1 · (0 ∪ 1)* · ε	start→accept = 1*00*1(0 ∪ 1)*

Por eso el programa muestra **(1)*0(0)*1((0 ∪ 1))***: es la misma expresión, pero **toString()** envuelve cada cerradura en paréntesis, y la unión ya trae los suyos. Solo es cosmético.



Ejemplo 2 — Resultado de «Reducir» en el programa

Comprobación: evaluar y simular. EvaluadorExpresionRegular compila "^" + toJavaRegex() + "$" y usa **matches()**. El botón «Verificar equivalencia» corre la misma cadena en el AFD y en la regex y compara los dos veredictos.

Cadena	AFD	Regex	¿Coinciden?
1011	acepta	acepta	sí
1101	acepta	acepta	sí
111	rechaza	rechaza	sí
0	rechaza	rechaza	sí

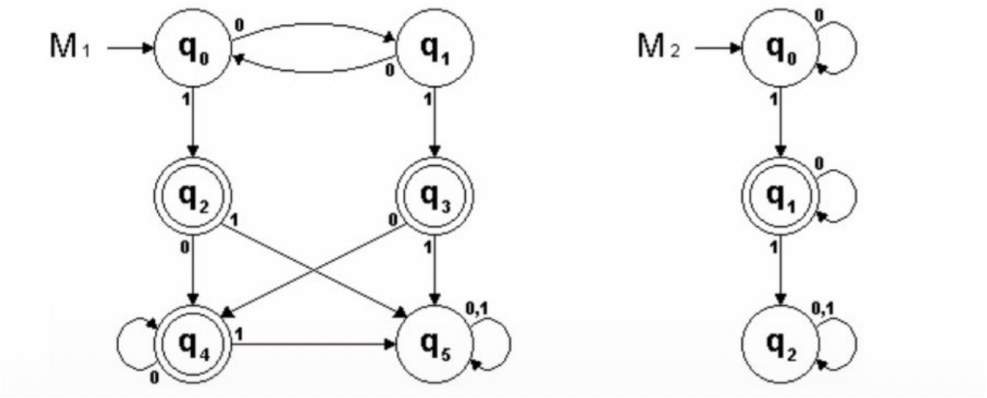
Además se probaron las 8191 cadenas binarias de hasta 12 caracteres, y el AFD y la regex no difieren en ninguna. Recorrido de 1011 en **SimuladorAnimado**: q0 --1--> q0 --0--> q1 --1--> q2 --1--> q2, y como q2 es final, queda ACEPTADA. El último paso usa la arista 0,1, y es justo donde antes fallaba la comparación con **equals**.

Minimización (extra, Clase 6)

Las particiones se representan como **List<Set<Estado>>** (cada Set es una clase de equivalencia), refinadas comparando la "firma" de destino de cada estado con un **Map<List<Integer>, Set<Estado>>** que agrupa estados con firma idéntica.

Verificación de equivalencia (extra, Clase 6) — ejemplo resuelto: M1 vs M2

Se explora el autómata producto de forma perezosa con BFS, usando un **record Par(Estado a, Estado b)** como clave de visitados; se detiene en cuanto encuentra un par donde uno es final y el otro no (esa es la cadena distinguidora), sin construir el producto completo en memoria. A continuación se muestra el ejemplo trabajado con dos autómatas guardados en archivos .afd y comparados con el botón «Comparar» (sección 3.7): el programa responde que son equivalentes, y aquí se explica por qué.



Ejemplo 3 — M1 (izquierda, 6 estados; finales q2, q3 y q4) y M2 (derecha, 3 estados; final q1)

El lenguaje de los dos. Ambos aceptan las cadenas con **un solo 1** y cualquier cantidad de ceros alrededor: 0*10*. En **M2**, q0 repite 0, el 1 lleva a q1 (final), q1 repite 0, y el segundo 1 lleva a la trampa q2. En **M1**, q0 y q1 solo cuentan si los ceros son pares o impares: el primer 1 lleva a q2 o q3 (finales), los 0 siguientes a q4 (final) y un segundo 1 a la trampa q5.

A mano, método 1: minimizar M1. Si el AFD mínimo de M1 resulta igual a M2, son equivalentes, porque el mínimo de un lenguaje es único.

Ronda	Partición	Qué pasa
Inicial	{q2,q3,q4} {q0,q1,q5}	Finales aparte de no finales.
1	{q2,q3,q4} {q0,q1} {q5}	q0 y q1 van con 0 a {q0,q1} y con 1 a los finales. q5 va con todo a no finales, así que se separa.
2	sin cambios	q2, q3 y q4 van con 0 a un final y con 1 a q5. Todo es estable.

Quedan tres clases: A = {q0,q1}, B = {q2,q3,q4} y C = {q5}. A repite con 0 y con 1 va a B; B repite con 0 y con 1 va a C; C es la trampa. Eso es M2 con otros nombres.

A mano, método 2: autómata producto. Se parte del par (q0 de M1, q0 de M2) y se avanza con 0 y 1. En cada par nuevo se compara si los dos lados son finales o no.

Par (M1, M2)	Cadena que llega	¿Final M1?	¿Final M2?	¿Coinciden?
(q0, q0)	ε	no	no	sí

Par (M1, M2)	Cadena que llega	¿Final M1?	¿Final M2?	¿Coinciden?
(q1, q0)	0	no	no	sí
(q2, q1)	1	sí	sí	sí
(q3, q1)	01	sí	sí	sí
(q4, q1)	10	sí	sí	sí
(q5, q2)	11	no	no	sí

Hay 6 pares alcanzables y en ninguno difieren. Si en algún par uno fuera final y el otro no, la cadena de esa fila sería la cadena distinguidora.

En el código: VerificadorEquivalencia

En la tabla a mano	En verificar(m1, m2)
Los dos autómatas deben tener todas las transiciones	completar(alfabetoUnido) en ambos. Aquí no agrega nada, porque ya son completos.
Par inicial (q0, q0)	new Par(c1.getEstadoInicial(), c2.getEstadoInicial()). Si ya difiere, devuelve «ε (cadena vacia)».
Avanzar con 0 y con 1	Cola pendientes y un for sobre alfabetoUnido.
Columna «Cadena que llega»	El mapa caminoHasta, que guarda prefijo + simbolo.
Par ya visto, se salta	if (caminoHasta.containsKey(siguiente)) continue;
Comparar si son finales	destinoA.esFinal() != destinoB.esFinal() devuelve Resultado(false, nuevoCamino).
Se acabaron los pares	return new Resultado(true, null);

```
Map<Par, String> caminoHasta = new LinkedHashMap<>(); // cadena que llevo a cada par
caminoHasta.put(inicio, "");
Deque<Par> pendientes = new ArrayDeque<>();
pendientes.add(inicio);
while (!pendientes.isEmpty()) {
    Par actual = pendientes.poll();
    String prefijo = caminoHasta.get(actual);
    for (Character simbolo : alfabetoUnido) {
        Estado destinoA = destinoPor(c1, actual.a(), simbolo);
        Estado destinoB = destinoPor(c2, actual.b(), simbolo);
        Par siguiente = new Par(destinoA, destinoB);
        if (caminoHasta.containsKey(siguiente)) continue;
        String nuevoCamino = prefijo + simbolo;
        caminoHasta.put(siguiente, nuevoCamino);
        if (destinoA.esFinal() != destinoB.esFinal())
            return new Resultado(false, nuevoCamino);
        pendientes.add(siguiente);
    }
}
return new Resultado(true, null);
```

En el código: MinimizadorAFD

En la tabla a mano	En MinimizadorAFD
Todo AFD con transiciones completas	original.completar(original.getAlfabeto())
Quitar lo inalcanzable	obtenerConexos(): búsqueda en anchura desde q0.
Partición inicial: finales y no finales	particionInicial()

En la tabla a mano	En MinimizadorAFD
Rondas hasta que nada cambie	refinar(): cada estado recibe una «firma», la clase a la que llega con cada símbolo; los de firma distinta se separan.
Una clase = un estado del mínimo	construirAutomataMinimo(): estados C0, C1… y símbolos paralelos unidos como 0,1.

El resultado del programa para M1 coincide con la tabla a mano: 3 estados. C1 es el inicial (A), C0 es el final (B) y C2 es la trampa (C), con C1→C1 = 0, C1→C0 = 1, C0→C0 = 0, C0→C2 = 1 y C2→C2 = 0,1. El inicial no es C0 porque la partición lista primero los finales. Al comparar el mínimo de M1 contra M2, el verificador también responde que son equivalentes.

El caso contrario: no equivalentes. Si en M2 se quita la marca de final de q1, el par (q2, q1) pasa a ser (final, no final) y es el primero que difiere. El programa responde «NO son equivalentes» con la cadena distinguidora **1**: M1 la acepta y M2 la rechaza.

Formato de los archivos .afd. Cada autómata se guarda como texto: una sección de estados (id, posición, inicial, final, etiqueta) y una de transiciones (origen, destino, símbolo). Ejemplo con M2 (las posiciones son ilustrativas):

```
ESTADOS
q0|60.0|60.0|true|false|q0
q1|220.0|60.0|false|true|q1
q2|380.0|60.0|false|false|q2
TRANSICIONES
q0|q0|0
q0|q1|1
q1|q1|0
q1|q2|1
q2|q2|0,1
```

Complemento (extra, Clase 6) — ejemplo resuelto: complemento de M2

El complemento acepta justo las cadenas que M2 rechaza. A mano: se conservan los estados y las transiciones, y los finales pasan a ser Q − F = {q0, q2}. Como M2 ya tiene todas sus transiciones, no hace falta agregar una trampa.

```
public Automata transformar(Automata original) { // GeneradorComplemento
    Automata complemento = original.completar(original.getAlfabeto()); // funcion total
    complemento.setNombre(original.getNombre() + "_complemento");
    for (Estado e : complemento.getEstados()) {
        e.marcarComoFinal(!e.esFinal());              // F' = Q - F
    }
    return complemento;
}
```

Cadena	Cantidad de 1	M2	Complemento de M2
ε	0	rechaza	acepta
0	0	rechaza	acepta
1	1	acepta	rechaza
10	1	acepta	rechaza
11	2	rechaza	acepta
101	2	rechaza	acepta

El complemento acepta las cadenas con ningún 1 o con dos o más. Se comprobó con todas las cadenas de hasta 12 caracteres. Completar primero es necesario: sin la trampa, una cadena que «se cae» quedaría rechazada tanto por el autómata como por su complemento.

5. Conclusiones

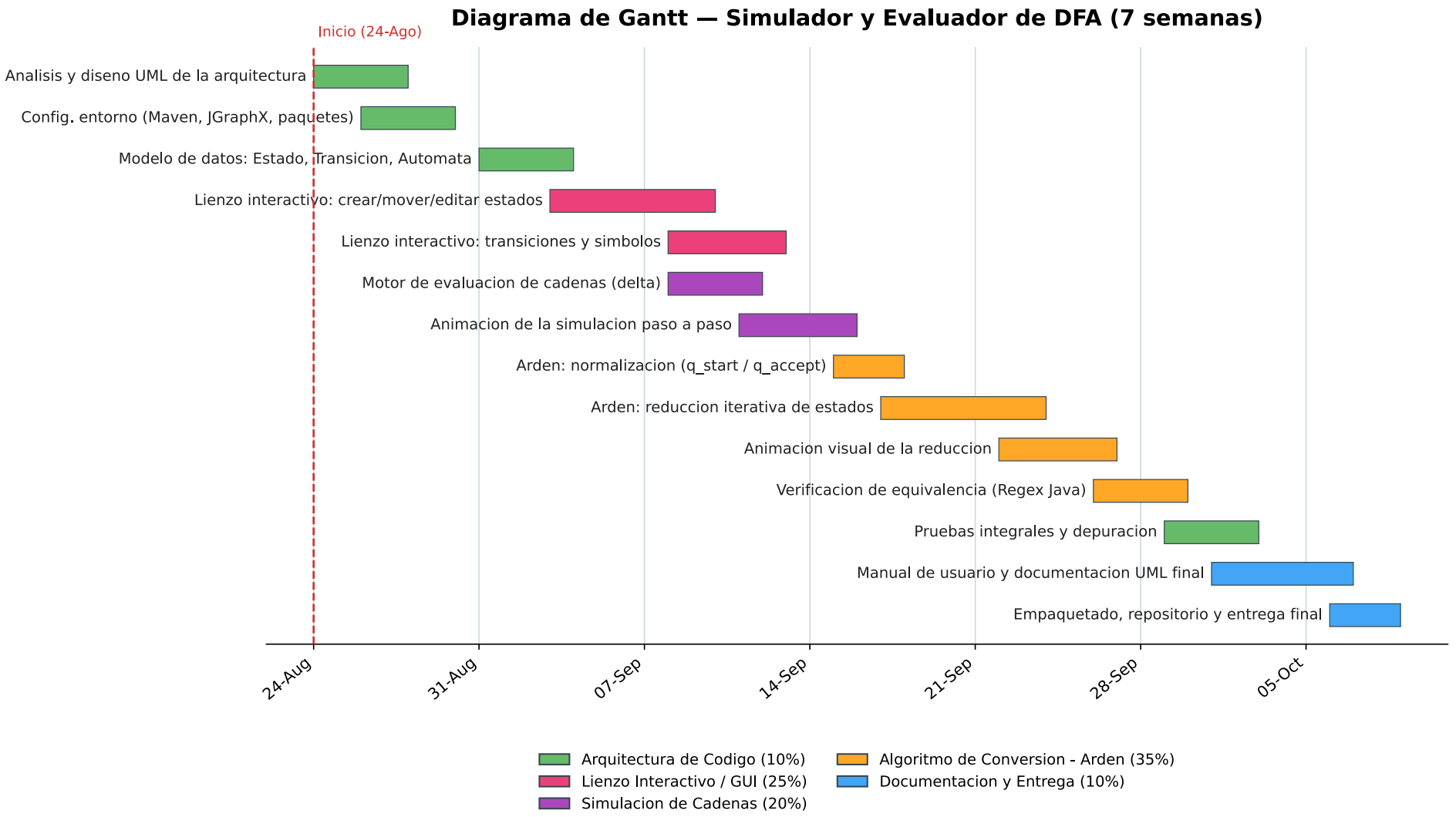
El sistema desarrollado cumple con los cuatro requerimientos funcionales establecidos: un lienzo interactivo para construir AFD gráficamente, un motor de simulación con recorrido animado, un algoritmo de eliminación de estados que aplica el Lema de Arden para obtener la expresión regular equivalente, y un módulo de verificación de equivalencia entre el AFD y dicha expresión. Adicionalmente, se incorporaron funcionalidades de cierre de lenguajes regulares (complemento, minimización y comparación de autómatas) correspondientes a los temas de la Clase 6, así como un selector de apariencia (Claro/Oscuro). El empaquetado del proyecto con Maven (BUILD SUCCESS) y la ejecución independiente del archivo .jar confirman que la arquitectura del software y el gestor de dependencias funcionan correctamente fuera del entorno de desarrollo.

Como se documentó en la sección 3.6, la verificación de equivalencia detectó inicialmente un caso de prueba en el que el AFD y la expresión regular generada no coincidían. La sección 4.3 explica, con el ejemplo resuelto a mano y la traza del código, que el origen estaba en cómo se interpretaban las etiquetas de transición con varios símbolos (por ejemplo, "0,1"), y cómo **EvaluadorExpresionRegular** y la unión de símbolos en **ReductorArden** resuelven ese caso correctamente.

6. Anexos

Anexo A presenta la planificación temporal del proyecto (diagrama de Gantt) y el Anexo B presenta la estructura de paquetes Java sobre la que se organiza el código fuente.

Anexo A — Diagrama de Gantt del Proyecto (7 semanas)



Planificación del proyecto por etapas: arquitectura de código, lienzo interactivo/GUI, simulación de cadenas, algoritmo de conversión (Arden) y documentación/entrega, distribuidas a lo largo de 7 semanas a partir del 24 de agosto.

Anexo B — Estructura de Paquetes

El código fuente se organiza en el paquete raíz **com.mycompany.automatas**, con la clase **Main** como punto de entrada, y cinco subpaquetes que reflejan la separación MVC del proyecto: **view** (interfaz gráfica), **controller** (orquestación de la aplicación), **model** (representación del autómata), **algorithm** (reducción de Arden y extras de Clase 6) y **simulation** (motor de simulación animada).

Estructura de Paquetes — com.mycompany.automatas

